



IEEE
**Computational
Intelligence**
Society



Universidade de Brasília
IEEE Student Branch

Trainee CIS 2026

Período 3 - LLMs

Conteúdos

1. Fundamentos dos Transformers e Atenção
 - a. [The Illustrated Transformer \(Jay Alammar\)](#) - Artigo visual explicando a arquitetura Transformer e o mecanismo de atenção.
 - b. [Attention Is All You Need \(Vaswani et al., 2017\)](#) - Paper original que introduziu a arquitetura Transformer.
 - c. [Transformers, the tech behind LLMs | Deep Learning Chapter 5 \(1Blue3Brown\)](#) - Vídeo explicando os conceitos fundamentais por trás dos Transformers e LLMs.
 - d. [Transformer Neural Networks, ChatGPT's foundation, Clearly Explained!!! \(StatQuest\)](#) - Vídeo explicando a arquitetura Transformer de forma intuitiva.
2. Embeddings e Representação Vetorial
 - a. [O que são Word Embeddings? \(Under The Hood\)](#) - Introdução aos embeddings e representação vetorial de palavras.
 - b. [Word Embedding and Word2Vec, Clearly Explained!!! \(StatQuest\)](#) - Explicação sobre Word2Vec e embeddings.
3. Tokenização e Pré-processamento
 - a. [Fast Tokenizer Superpowers \(Hugging Face\)](#) - Tutorial explicando tokenização eficiente utilizando Hugging Face.
4. Implementação Prática
 - a. [Let's Build GPT: From Scratch, in Code, Spelled Out \(Andrej Karpathy\)](#) - Implementação prática de um modelo GPT do zero.
 - b. [Hugging Face Course](#) - Curso introdutório sobre Transformers, tokenização, pipelines e fine-tuning de modelos open-source.
5. Retrieval Augmented Generation (RAG)
 - a. [RAG From Scratch \(LangChain\)](#) - Uma playlist fundamental desenvolvida pela equipe da LangChain que constrói o RAG desde os conceitos mais básicos (chunking e embeddings no Python puro) até estratégias avançadas de recuperação.
 - b. [Pipeline de RAG](#) - Breve artigo detalhando o fluxo de um RAG, com exemplos em código.
 - c. [RAG From Scratch \(Hugging Face\)](#) - Tutorial de como implementar um RAG usando apenas o ollama

Atividade Prática

Obs. As atividades abaixo podem ser desenvolvidas no ambiente em que o autor(a) julgar melhor. Porém, deve ser enviado o código deve ser hospedado em um repositório do github e o seu link deve ser submetido via [formulário](#).

1. Atividade obrigatória:
 - a. Construir um pipeline de RAG (Retrieval-Augmented Generation) simples utilizando o dataset [wikipedia-PT/BR](#) (via biblioteca datasets do Hugging Face) como base de conhecimento, respondendo perguntas em linguagem natural sobre os artigos indexados;
2. Atividade opcional:
 - a. Realizar o pré-processamento dos documentos, dividindo os artigos em chunks de tamanho fixo (ex: 256 ou 512 tokens) com sobreposição (overlap), e analisar o impacto do tamanho do chunk na qualidade das respostas recuperadas;
 - b. Gerar embeddings dos chunks utilizando um modelo open-source (ex: all-MiniLM-L6-v2 via sentence-transformers), indexá-los com FAISS e realizar buscas por similaridade para recuperar os trechos mais relevantes dado um query;
 - c. Utilizar um LLM open-source (ex: google/flan-t5-base ou TinyLlama) como gerador, passando os chunks recuperados como contexto no prompt e avaliando qualitativamente se as respostas geradas são mais precisas do que as do modelo sem recuperação (closed-book);